

Sistema de Gestão de Condomínio

Relatório do Projeto Prático (Opção 1: Sistema de Informação)

Vivian Azevedo · Clécio Muniz · Sâmia Freitas · Beatriz Luna

Repositório: github.com/vivianazevedo/edoo-adm-condominio · Página: vivianazevedo.github.io/edoo-adm-condominio

1. Introdução

Este relatório descreve o sistema de gestão de condomínio desenvolvido como projeto prático da disciplina, escolhendo a opção 1 (sistema de informação com CRUD conectado a banco de dados). O sistema administra **apartamentos, moradores, funcionários, visitantes, áreas comuns e reservas**, em **C++17** com banco **SQLite**, build em **CMake** e duas formas de uso sobre o mesmo núcleo: uma interface gráfica em **Qt 6 Widgets** (bônus) e menus no terminal. O foco do relatório é a arquitetura e o uso de orientação a objetos; o que está descrito aqui corresponde ao código presente na branch `main` do repositório.

2. Descrição do sistema

Funcionalidades implementadas, todas com persistência no SQLite:

- **Apartamentos:** CRUD completo (bloco e número únicos).
- **Moradores:** cadastro vinculado a um apartamento, com tipo de ocupação (proprietário, inquilino ou dependente), listagem geral ou por apartamento, edição e remoção.
- **Funcionários:** CRUD completo, com cargo (porteiro, zelador, faxineiro ou administrador), turno e data de admissão.
- **Visitantes e visitas:** cadastro de visitantes, registro de entrada e saída por um porteiro e histórico por apartamento.
- **Áreas comuns:** salão de festas, piscina e churrasqueira, com capacidade, taxa-base e horário de funcionamento.
- **Reservas:** criação e cancelamento com validação de regras e cálculo automático do valor.

As principais regras de negócio ficam na camada de serviço e no modelo:

Módulo	Regras
Reservas	Não reservar em data/horário que já passou; sem sobreposição com outra reserva ativa na mesma área e data; respeito à capacidade, ao horário de funcionamento e à duração máxima (salão 8 h, piscina 2 h, churrasqueira 4 h); cancelamento só com 24 h de antecedência e uma única vez.
Visitas	Somente funcionário com cargo de porteiro registra entrada; cada visita recebe uma só saída, e a saída não pode ser anterior à entrada.
Cadastros	CPF válido (dígitos verificadores) e único entre todas as pessoas; bloco e número de apartamento únicos; não se removem apartamento com moradores, morador com reservas, funcionário com visitas nem área com reservas.

3. Arquitetura em camadas

O sistema é dividido em camadas para que cada parte seja desenvolvida e testada separadamente. O núcleo (`condominio_core`) não depende do Qt; o terminal e a interface gráfica usam os mesmos serviços e o mesmo banco. As telas nunca executam SQL: o fluxo de uma operação é **interface** → **serviço** → **repositório** → **SQLite**.

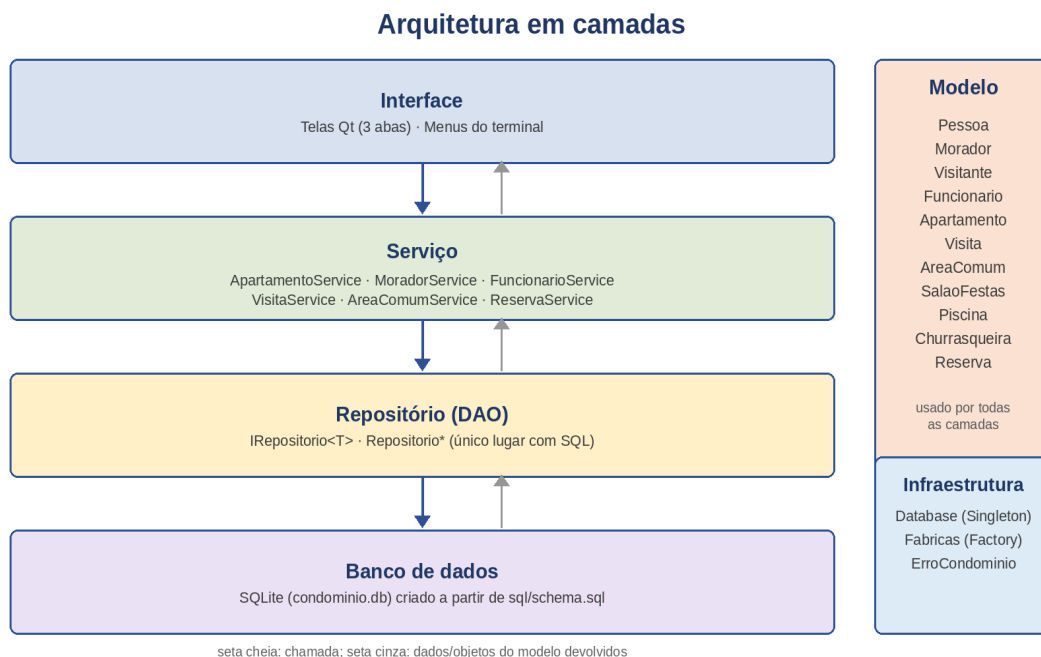


Figura 1 – Arquitetura em camadas e fluxo de uma operação.

Camada	Responsabilidade	Classes principais
Interface	Telas Qt (3 abas) e menus do terminal; só encaminham ações e exibem erros	TelaMoradores, TelaReservas, TelaVisitas, MenuPrincipal, MenuMoradores, MenuReservas, MenuVisitas
Serviço	Regras de negócio e funções do usuário final	ApartamentoService, MoradorService, FuncionarioService, VisitaService, AreaComumService, ReservaService
Repositório (DAO)	CRUD, isolando o SQL	IRepositorio<T>, RepositorioApartamento, RepositorioPessoa, RepositorioVisita, RepositorioAreaComum, RepositorioReserva
Infraestrutura	Conexão, exceções e fábricas	Database, ErroCondominio (e filhas), FabricaPessoa, FabricaAreaComum, FabricaPorTipo<T>
Modelo	Entidades usadas por todas as camadas	Pessoa, Morador, Visitante, Funcionario, Apartamento, Visita, AreaComum (+3 subclasses), Reserva

O repositório tem a organização de pastas `include/` e `src/` (modelo, repositorio, servico, infra, interface, terminal), `sql/` (schema e dados de exemplo), `tests/` (testes via CTest) e `docs/`. O SQLite vem embutido em `third_party/sqlite`, então não há instalação separada do banco.

4. Modelo de classes

O diagrama da Figura 2 mostra as hierarquias de herança, as interfaces e as associações. Há duas árvores de herança do domínio (Pessoa e AreaComum) e uma de exceções; Apartamento, Reserva e Visita não usam herança e se ligam às demais por associação (identificadores e ponteiros não proprietários).

Diagrama de classes - Sistema de Gestão de Condomínio

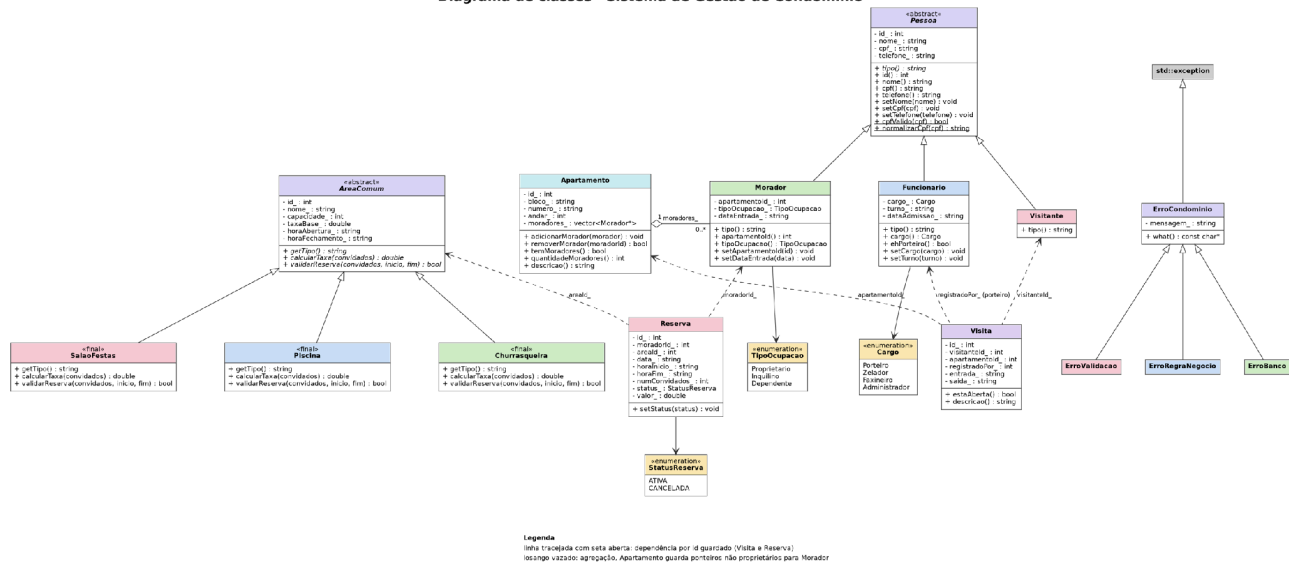


Figura 2 – Diagrama de classes final (docs/diagrama-classes.png).

Hierarquia	Descrição
Pessoa (abstrata)	Declara tipo() como virtual puro. Subclasses: Morador (enum TipoOcupacao), Visitante e Funcionario (enum Cargo).
AreaComum (abstrata)	Declara getTipo(), calcularTaxa() e validarReserva() como virtuais puros. Subclasses: SalaoFestas , Piscina e Churrasqueira (classes final).
ErroCondominio	Herda de std::exception. Subclasses: ErroValidacao (dado inválido), ErroRegraNegocio (regra quebrada) e ErroBanco (falha no SQLite). O terminal imprime; o Qt mostra em diálogo.
IRepositorio<T>	Interface genérica (classe abstrata, template) com inserir, buscarPorId, listar, atualizar e remover. Há ainda IRepositorioAreaComum e IRepositorioReserva, com consultas específicas.
Sem herança	Apartamento (guarda lista de ponteiros não proprietários para seus moradores), Reserva (liga morador e área, com enum StatusReserva) e Visita (liga visitante, apartamento e porteiro).

5. Conceitos de POO aplicados

Conceito	Onde e como foi usado
Classes e objetos	Todas as entidades do domínio, serviços, repositórios e telas; um par .h/.cpp por classe.
Herança	Pessoa → Morador/Visitante/Funcionario; AreaComum → SalaoFestas/Piscina/Churrasqueira; ErroCondominio → três exceções.
Classes abstratas e virtuais puros	Pessoa, AreaComum e IRepositorio<T>, sempre com destrutor virtual para apagar corretamente pelo ponteiro da base.
Polimorfismo (virtual/override)	Cada área calcula a taxa e valida a reserva à sua maneira; cada pessoa informa seu tipo(). O serviço trabalha só com o ponteiro da base.
Encapsulamento e modificadores de acesso	Atributos private/protected, getters e setters com validação (CPF, datas, horários, capacidade); construtores validam ao criar o objeto.
Ponteiros e referências	unique_ptr nos objetos devolvidos por fábricas e repositórios; shared_ptr nas dependências de ReservaService; referências (IRepositorio<Pessoa>&) em VisitaService; const& nos parâmetros; Morador* não proprietário em Apartamento.
Composição e associação	Reserva liga Morador e AreaComum; Visita liga Visitante, Apartamento e Funcionario (porteiro); Apartamento mantém seus moradores.

Conceito	Onde e como foi usado
Tratamento de exceções	Hierarquia própria; os serviços lançam, a interface captura e exhibe.

5.1 Polimorfismo em áreas comuns e reservas

A classe abstrata `AreaComum` define o contrato e cada subclasse o reescreve com `override`. Exemplo da `Piscina`, que é gratuita, aceita até 4 convidados e no máximo 2 horas:

```
double Piscina::calcularTaxa(int convidados) const {
    if (convidados < 0 || convidados > getCapacidade() || convidados > 4)
        throw ErroValidacao("Numero de convidados invalido para piscina");
    return 0.0;
}
bool Piscina::validarReserva(int convidados, const std::string& inicio,
                             const std::string& fim) const {
    return convidados <= 4 && validarPeriodo(convidados, inicio, fim, 2 * 60);
}
```

O `SalaoFestas` cobra taxa-base mais R\$ 3,00 por convidado e a `Churrasqueira` cobra R\$ 40,00 fixos. No `ReservaService::criar`, o serviço obtém a área pelo repositório (um `unique_ptr<AreaComum>`) e chama `area->validarReserva()` e, ao final, `area->calcularTaxa()`, sem saber qual é a subclasse. Entre as duas chamadas, ele verifica a sobreposição de horários com as reservas ativas da mesma área e data (`horaInicio < fimExistente && horaFim > inicioExistente`).

5.2 Fluxo completo de uma reserva

Para mostrar como as camadas e os conceitos trabalham juntos, o caminho de uma reserva criada pela interface é:

- **Interface:** `TelaReservas` lê o formulário e chama `ReservaService::criar(...)`; qualquer exceção é capturada e exibida em `QMessageBox`.
- **Serviço:** valida o objeto `Reserva` (datas, horários, ids), recusa datas passadas, busca a área pelo repositório e chama `validarReserva()` (polimorfismo).
- **Serviço:** consulta `buscarPorAreaEData` e recusa sobreposição com reservas ativas (RN01); calcula o valor com `calcularTaxa()`.
- **Repositório:** `RepositorioReserva` insere no SQLite com comando preparado; se a chave estrangeira falhar, o serviço traduz o `ErroBanco` em `ErroRegraNegocio`.

6. Design patterns

Pattern	Implementação
Singleton + RAII	<code>Database::instancia()</code> devolve a única conexão SQLite do processo. O construtor abre o banco, ativa <code>PRAGMA foreign_keys = ON</code> e cria as tabelas a partir de <code>sql/schema.sql</code> se o banco estiver vazio; o destrutor fecha a conexão; cópia e movimentação são desabilitadas. Escolha adequada a um aplicativo local e monousuário.
Repository (DAO)	<code>IRepositorio<T></code> define o CRUD e cada entidade tem sua implementação, que é o único lugar com SQL (comandos preparados com parâmetros). Em <code>RepositorioPessoa</code> , gravar os dados comuns e os do subtipo é feito em transação. <code>buscarPorId</code> devolve <code>unique_ptr<T></code> (<code>nullptr</code> se não existe).
Factory	<code>FabricaPorTipo<Base></code> associa o texto salvo no banco a uma função criadora e rejeita tipos não registrados. <code>FabricaPessoa</code> registra morador, visitante e funcionario; <code>FabricaAreaComum</code> registra <code>SalaoFestas</code> , <code>Piscina</code> e <code>Churrasqueira</code> . Assim, ao ler do banco, o repositório devolve a subclasse correta por ponteiro da base, preservando o polimorfismo.

O padrão `Observer`, previsto como extra, não foi implementado: as telas Qt são atualizadas ao trocar de aba.

7. Modelo de dados

O esquema tem sete tabelas: `apartamento`, `pessoa`, `morador`, `funcionario`, `area_comum`, `reserva` e `visita`. Visitantes ficam na tabela `pessoa` (coluna `tipo` distingue morador, visitante e funcionário);

morador e funcionario usam pessoa_id como PK e FK. Cada visita é um registro separado, para preservar o histórico. Chaves estrangeiras e restrições de unicidade (CPF; bloco e número) garantem a integridade no banco.

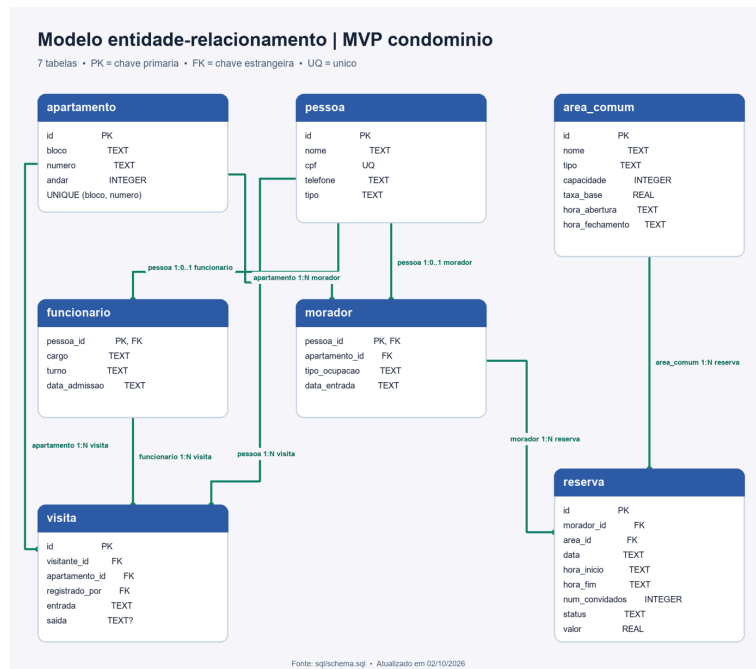


Figura 3 – Diagrama entidade-relacionamento (docs/diagrama-er.png).

8. Interface gráfica e testes

A interface Qt 6 (condominio_gui) tem três abas: **Moradores e Aptos**, **Áreas e Reservas** e **Portaria e Visitas** (onde também ficam os funcionários). As telas usam os serviços do núcleo e mostram os erros de regra de negócio em caixas de diálogo. O terminal (condominio_terminal) oferece menus equivalentes.

Os testes automáticos rodam via CTest sobre um SQLite em memória e cobrem banco, repositórios, fábricas, serviços, correções de bugs, fluxos da interface (plugin Qt offscreen) e dez cenários de integração (D02) com os serviços reais: de cadastros e vínculo morador–apartamento à rejeição de reserva conflitante, fora do horário ou acima da capacidade, e ao cancelamento com e sem 24 h de antecedência. Segundo o README, são 14 testes (13 sem Qt).

Como compilar e executar

Requisitos: compilador C++17 e CMake 3.16 ou superior (Qt 6 Widgets só para a interface gráfica; sem ele o CMake compila o núcleo, o terminal e os testes). Comandos básicos:

```
cmake -S . -B build -DCMAKE_PREFIX_PATH="<pasta do Qt>"
cmake --build build
cd build && ./condominio_gui          # ou ./condominio_terminal
ctest --test-dir build --output-on-failure
```

O banco condominio.db é criado automaticamente na primeira execução; sql/seed.sql traz dados fictícios opcionais para demonstração.

Limitações e trabalhos futuros

- A consulta de reservas por área e data existe no serviço e nos testes, mas não aparece na interface nem no terminal.
- A edição de área comum não altera tipo nem horário de funcionamento; o terminal não edita nem remove áreas.
- Ficaram de fora os extras: avisos importantes, subclasses Porteiro e Zelador, Observer, relatórios extras, testes com doctest e Supabase.

- Fora do escopo: pagamento, login e senha, acesso simultâneo e versão web/mobile (o porteiro é escolhido no formulário).

9. Divisão do trabalho

Resumo da contribuição de cada integrante, conforme o histórico de commits e Pull Requests do repositório:

Integrante	Contribuições
Vivian Azevedo	Hierarquia Pessoa (Pessoa, Morador, Visitante, Funcionario), Apartamento, Visita e exceções do sistema; repositórios de Apartamento, Pessoa e Visita; ApartamentoService, MoradorService e FuncionarioService; menu de moradores; diagrama de classes final; tema de cores da interface Qt; README, página github.io, comentários do código e consolidação do relatório.
Clécio Muniz	Infraestrutura: Database (Singleton/RAII), schema.sql e seed.sql, contrato IRepository<T>, fábricas, CMake e organização dos testes; modelos de áreas comuns e reservas; integração das áreas e reservas ao SQLite; integração da interface Qt aos serviços; diagrama ER, cenários de integração D02, texto de arquitetura e roteiro do vídeo.
Sâmia Freitas	Esqueleto da interface Qt com navegação por abas; menu principal do terminal; interfaces de repositório e serviços de visita e reserva; correções de datas e regras de negócio.
Beatriz Luna	Repositórios de AreaComum e Reserva; AreaComumService e ReservaService com as regras de reserva (conflito, capacidade, horário, cancelamento); submenu de áreas e reservas no terminal; documentação de polimorfismo e regras de reserva.

10. Referências

- Slides do professor: "7. Project with OOP" (CIN0135, 2026.2).
- KIRCH-PRINZ, U.; PRINZ, P. A Complete Guide to Programming in C++. 2002.
- Documentação do Qt, do SQLite e do CMake.